

How Computers Work · Section 8

How Programs Are Built and Run

By TOUCH DEN · November 11, 2026

Contents

What You Will Learn 1

Before You Start 1

1. From Source Code to a Running Program 1

2. Compiled vs. Interpreted Languages 2

3. The Build Pipeline: Compile and Link 3

4. Machine Code and Assembly 3

5. Libraries: Static and Dynamic 4

6. What an Executable File Is (ELF, PE, Mach-O) 5

7. Why This Matters for Reversing and Malware 6

Common Mistakes 6

Summary 7

Check Your Understanding 7

Practice 8

What You Will Learn

- How source code becomes a running program.
- The difference between compiled and interpreted languages.
- The build pipeline: compiling and linking.
- Machine code and assembly.
- Libraries, static and dynamic.
- What an executable file is (ELF, PE, Mach-O).
- Why all of this matters for reverse engineering and malware analysis.

Before You Start

Read lessons 1, 2, and 6. You know everything is bytes, a program is instructions in memory, and the CPU fetches and executes them. This lesson shows where those instructions come from: how a human-written program becomes bytes the CPU can run. It uses a tiny **C** program, compiled with Clang; C is introduced from zero, just enough to see what compiling does. The outputs are from real runs on this machine.

---

1. From Source Code to a Running Program

People do not write the raw numeric instructions a CPU executes; that would be unbearable. Instead, they write **source code**: instructions in a programming language that humans can read, like C, Python, or JavaScript. Source code is just **text** (lesson 3), a file of characters, and the CPU cannot run text. Something must turn that human-readable text into the numeric machine instructions the CPU understands (lesson 2).

Here is a tiny, complete C program. Do not worry about the exact syntax; read it like English. It prints a line, and adds two numbers:

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("Mina Coffee\n");
    printf("2 lattes = %d coffees\n", add(1, 1));
    return 0;
}
```

That is source code: readable text. On its own, the CPU cannot run it. To actually run, it must be turned into machine instructions. There are two main ways to make that happen, compiling and interpreting, and the difference shapes a lot of security.

2. Compiled vs. Interpreted Languages

There are two strategies for running source code.

- **Compiled languages** (like C, C++, Rust, Go) are translated *ahead of time*, in one step, into a file full of machine code, the **executable**. You run the executable directly; the original source is not needed anymore, and not present on the user's machine. This is like translating a whole book into another language once, then handing out the translation.
- **Interpreted languages** (like Python, JavaScript) are translated *as they run*, line by line, by another program called the **interpreter**. You ship the source code, and the user's interpreter reads and runs it on the spot. This is like a live interpreter translating a speech sentence by sentence.

The difference matters for security in a very practical way:

- A **compiled** program ships as machine code, not readable source. To understand what it does, you must **reverse engineer** it (course 17): read the machine instructions. This is the world of analyzing a suspicious `.exe`, where the source is nowhere to be found.
- An **interpreted** program ships as source (or something close to it), so its logic is often much easier to read, sometimes you can just open it. Malicious scripts (in Python, JavaScript, PowerShell) are common precisely because they are easy to write and deliver, and analyzing them is reading code rather than machine instructions.

Many modern languages blend the two (Java and C# compile to an in-between "bytecode" that a runtime then runs), but the core distinction holds: **compiled code hides its logic in machine code; interpreted code ships its logic as readable source.**

3. The Build Pipeline: Compile and Link

Turning C source into an executable actually takes a few steps, together called **building** the program. Run with Clang, our program builds and runs like this:

```
$ clang -o greet greet.c
$ ./greet
Mina Coffee
2 lattes = 2 coffees
```

That one `clang` command hides two important stages:

1. **Compiling:** the compiler translates your source code into machine code, producing an **object file** of raw instructions for your functions. Each source file is compiled to its own object file.
2. **Linking:** the **linker** combines your object file(s) with any **library** code you used (here, the `printf` function, which you did not write, topic 5) into one finished executable that can actually run.

So "compile" loosely means the whole journey, but precisely, compiling makes the machine code for *your* code, and linking stitches it together with everything else into a runnable file. If you have ever seen a "linker error" versus a "compiler error", this is the difference: the compiler could not understand your code, versus the pieces could not be joined together.

You do not need to run builds yet, that is for the programming courses, but recognize the shape: **source code → compile → object files → link (with libraries) → executable.** The reverse of this journey, taking an executable back toward readable form, is exactly what reverse engineering attempts.

4. Machine Code and Assembly

What does the compiler actually produce? **Machine code**: the raw numeric instructions the CPU fetches and executes (lesson 4). Each instruction is a pattern of bytes that means something specific to the processor, "add these two registers", "jump to this address". Machine code is what the executable file contains, and it is not readable by humans in any practical way, it is just bytes.

Because raw machine-code bytes are unreadable, there is a human-friendly version called **assembly language**: a text form where each machine instruction is written as a short mnemonic, like `add`, `mov` (move), `cmp` (compare), or `jmp` (jump). Assembly is almost a one-to-one translation of machine code, just readable. When a reverse engineer studies a compiled program, the tool (a **disassembler**) turns the machine-code bytes back into assembly, and the analyst reads *that*.

You are not learning assembly here, that is the Reverse Engineering course (17). But understand the ladder:

- **Source code** (C, readable, high level) →
- **Machine code** (bytes, what the CPU runs) ↔
- **Assembly** (the readable text form of machine code).

The compiler goes down the ladder (source → machine code). The reverse engineer climbs partway back up (machine code → assembly), because the original source is gone. This is why reversing is hard and skilled work: you are reconstructing meaning from assembly, without the helpful names and structure the original author had.

5. Libraries: Static and Dynamic

Our tiny program used `printf` to print, but we never wrote `printf`. It came from a **library**: a collection of ready-made code that programs reuse instead of reinventing. Almost every real program leans on many libraries (for math, networking, graphics, encryption), which is why Security Fundamentals called an app "mostly other people's code." Libraries connect to a program in two ways:

- **Static linking:** the library's code is copied *into* the executable at build time. The program is self-contained (bigger, but needs nothing else).
- **Dynamic linking:** the library stays a separate file (a `.dll` on Windows, a `.so` on Linux, a `.dylib` on macOS), and the program loads it at run time. Many programs can share one copy, saving space and allowing the library to be updated independently.

This matters for security in several ways you will meet later:

- **Supply-chain risk** (Security Fundamentals, Secure Development): a vulnerability or a backdoor in a library affects every program that uses it. Much of modern security is tracking and patching library flaws.
- **DLL/shared-library hijacking:** because dynamically linked programs load libraries by name at run time, an attacker who plants a malicious library where the program looks can get their code loaded, a real attack technique.

- **Updating:** a dynamically linked library can be patched once to fix every program using it; a statically linked flaw must be rebuilt into each program.

So when you analyze or defend a program, part of the question is always: what libraries does it use, and are they trustworthy and up to date?

6. What an Executable File Is (ELF, PE, Mach-O)

An executable is not just a naked blob of machine code. It is a **structured file** in a standard format, with the machine code *plus* information the OS needs to run it: where the code goes in memory, which libraries to load, where execution starts, and so on. The format depends on the operating system:

- **ELF** on Linux (and Android).
- **PE** (Portable Executable, the familiar `.exe` and `.dll`) on Windows.
- **Mach-O** on macOS (and iOS).

You can see the format with the `file` command (lesson 7's magic bytes at work, executables have signatures too). Here is our compiled program on this machine, and, tellingly, its readable strings:

```
$ file greet
greet: Mach-O 64-bit executable arm64
$ strings greet | grep -i mina
Mina Coffee
```

Two lessons jump out. First, `file` identifies the executable's format from its content. Second, and importantly for security: even though the program is compiled machine code, the **text strings** inside it (like `Mina Coffee`) are still readable, the `strings` tool pulls them straight out. This is why one of the very first steps in analyzing any unknown program, malware included, is to run `strings` on it: web addresses, messages, file names, and clues often sit in plain sight inside a compiled binary. (This is the exact technique the mobile-security course used to read a secret out of an app.)

Understanding executable formats is the foundation of reverse engineering and malware analysis. A malware analyst looking at a suspicious `.exe` is reading a PE file: its structure, its libraries, its strings, and its machine code. Everything in this lesson is what they rely on.

7. Why This Matters for Reversing and Malware

Pulling the lesson together: the journey from source code to a running program is the exact ground that two major security fields stand on.

- **Reverse engineering** (course 17) runs this journey *backward*. Given only an executable, no source, the analyst uses a disassembler to turn machine code into assembly and reconstruct what the program does. Everything here, compiled vs interpreted, machine code vs assembly, executable formats, libraries, defines what that work involves.
- **Malware analysis** (course 29) applies reversing to malicious programs. Is this `.exe` a trojan? What does it do? The analyst checks its format (PE), its strings, its libraries, and its code, exactly the concepts of this lesson, often starting with the easy wins (`file`, `strings`) before the hard work of reading assembly.

Even on defense, this knowledge is everyday. Knowing that compiled programs hide their logic, that interpreted scripts reveal it, that strings leak clues, and that libraries carry risk, shapes how you analyze any unknown program you find on a system.

You now understand how a program comes to exist and run. Next, we step back to the very beginning of a computer's life, how it goes from a cold power-off state to a running operating system: the boot process.

Common Mistakes

1. Thinking the CPU runs source code. It runs machine code. Source code is text that must be compiled (ahead of time) or interpreted (as it runs) first.

2. Treating "compile" and "link" as the same. Compiling makes machine code for your code; linking joins it with libraries into a runnable executable. Their errors are different.

3. Assuming compiled programs reveal nothing. Strings, libraries, and structure are still visible. `strings` on a binary is a classic, fast first step in analysis.

4. Ignoring libraries. Most of a program is library code. A library's flaw is your program's flaw (supply-chain risk), and libraries can be hijacked.

5. Confusing machine code with assembly. Machine code is the bytes the CPU runs; assembly is its readable text form. Reverse engineers read assembly produced by a disassembler.

Summary

Term	Remember
Source code	Human-readable program text; the CPU can't run it directly
Compiled vs interpreted	Translated ahead of time into an executable vs translated line-by-line as it runs
Compile + link	Compile makes machine code for your code; link joins it with libraries
Machine code / assembly	The bytes the CPU runs / the readable text form of those bytes
Library	Reused code; static (copied in) or dynamic (separate file loaded at run time)
Executable format	ELF (Linux), PE/.exe (Windows), Mach-O (macOS); structured file + machine code
Reversing/malware	Run the journey backward; `strings` and `file` are the fast first steps

Check Your Understanding

1. Why can't the CPU run source code directly?
2. What is the key difference between a compiled and an interpreted language, and how does each affect analysis?
3. What do compiling and linking each do?
4. What is the difference between machine code and assembly, and which does a reverse engineer read?
5. What is the difference between static and dynamic linking, and name one security issue each relates to?
6. Name the executable format on Linux, Windows, and macOS.
7. Why is running `strings` on an unknown compiled program a useful first step?

Answers

1. The CPU runs numeric machine instructions, but source code is human-readable text. It must first be compiled (ahead of time) or interpreted (as it runs) into machine code.
2. A compiled language is translated ahead of time into a machine-code executable (source not shipped), so analysis means reverse engineering the machine code. An interpreted language ships its source and is translated as it runs, so its logic is often readable directly.
3. Compiling translates your source into machine code (object files); linking combines those with library code into one finished, runnable executable.

4. Machine code is the raw numeric instructions the CPU runs; assembly is a readable text form of those instructions, almost one-to-one. A reverse engineer reads assembly (produced by a disassembler).
5. Static linking copies the library into the executable (self-contained); dynamic linking keeps it a separate file loaded at run time (shared, updatable). Static relates to needing to rebuild each program to fix a flaw; dynamic relates to DLL/shared-library hijacking and shared updates.
6. ELF (Linux), PE/.exe (Windows), Mach-O (macOS).
7. Because readable text strings (web addresses, messages, file names) survive compilation and sit in plain sight inside the binary, giving quick clues about what the program does before the harder work of reading its code.

Practice

1. Decide for each whether it is **compiled** or **interpreted**, and what that means for reading its logic: (a) a Windows `.exe`; (b) a Python `.py` script; (c) a JavaScript file on a web page.
2. In your own words, explain the pipeline "source → compile → link → executable" to a friend, using the book-translation analogy.
3. If you can run a terminal with a C compiler, compile the `greet` program from topic 1 and run `strings` on the result. Which readable text from the program can you find inside the binary?

A Worked Example

1. (a) `.exe` is compiled machine code, so reading its logic means reverse engineering it. (b) a `.py` script is interpreted and shipped as source, so you can just open and read it. (c) JavaScript on a web page is interpreted and sent as source, so it is readable (anyone can view it in the browser).
2. "Writing source code is like writing a book in a language the CPU can't read. Compiling translates each chapter into the CPU's language. Linking binds those translated chapters together with the quoted passages from other books (libraries) into one finished volume, the executable, which the CPU can finally read and run."
3. After `clang -o greet greet.c`, running `strings greet` reveals readable text including `Mina Coffee` and the `2 lattes = %d coffees` format line, proving that text strings survive compilation and are easy to extract from a compiled binary.

Course mark icon from Material Symbols by Google, Apache License 2.0.